HOWARD Tips

1 HOWARD Tips

How to annotate a VCF file with a Parquet database?

Annotate a VCF file is simple using `annotation` tool. Number of threads and memory can be handled.

```
INPUT=tests/data/example.vcf.gz
OUTPUT=/tmp/example.annotated.vcf.gz
PARQUET=tests/databases/annotations/current/hg19/nci60.parquet
howard annotation \
   --input="$INPUT" \
   --output="$OUTPUT" \
   --annotations="$PARQUET" \
   --threads=8 --memory=4G
```

How to speedup annotation and reduce memory for a huge input VCF and/or a huge Parquet database?

For huge input VCF file and/or huge annotation Parquet database, and if it's the uniq process, two specific parameters can speedup annotation and reduce memory usage:

- `fast`: speedup execution to focus on Parquet annotation without any other process
- `access`: read only strategy will not load input data but read the input file

The strategy consist on converting the input VCF file into Parquet, and then annotate it.

```
INPUT=tests/data/example.vcf.gz
OUTPUT=/tmp/example.annotated.vcf.gz
PARQUET=tests/databases/annotations/current/hg19/nci60.parquet
# 1. Convert input VCF file into Parquet
howard convert \
   --input="$INPUT" \
   --output="$OUTPUT.parquet"
   --access='RO' \
   --threads=2 --memory=4G
# 2. Annotate input Parquet file
howard annotation \
   --input="$OUTPUT.parquet" \
   --output="$OUTPUT" \
   --annotations="$PARQUET" \
   --access='RO' \
   --fast \
   --threads=2 --memory=4G
```

How to hive partitioning into Parquet a VCF format file?

In order to create a database from a VCF file, process partitioning highly speed up futher annotation process. Simply use HOWARD Convert tool with `--parquet_partitions` option. Format of input and output files can be any avaialbe format (e.g. Parquet, VCF, TSV).

This process is not ressource intensive, but can take a while for huge databases. However, using `--explode_infos` option (annotations in columns) requires much more memory for huge databases.

```
INPUT=~/howard/databases/dbsnp/current/hg19/b156/dbsnp.vcf.gz
OUTPUT=~/howard/databases/dbsnp/current/hg19/b156/dbsnp.partition.parquet
```

```
PARTITIONS="#CHROM" # "#CHROM", "#CHROM,REF,ALT" (for SNV file only)
howard convert \
   --input=$INPUT \
   --output=$OUTPUT \
   --parquet_partitions="#CHROM" \
   --threads=8
```

How to process a huge input VCF file?

To process a huge input VCF file efficiently, a strategy constits on partitioning it into multiple smaller Parquet files, process each partition individually, and then merge the results into a final output file. This approach helps manage memory usage and speeds up processing for large datasets. The `chunking` strategy is implemented into `process` tool:

```
# Initialize variables
input_file=tests/data/example.vcf
output_file=/tmp/example.test.vcf
chunking_size=2 # default 1000000
howard process \
    --input=$input_file \
    --output=$output_file \
    --calculations="VARTYPE" \
    --chunking_enable \
    --chunking_size=$chunk_size
```

This method ensures efficient handling of large files while maintaining flexibility for various processing tasks.

How to aggregate all INFO annotations from multiple Parquet databases into one INFO field?

In order to merge all annotations in INFO column of multiple databases, use a SQL query on the list of Parquet databases, and use `GROUP_CONCAT` duckDB function to aggretate values. This method can requires huge memory if Parquet files are huge.

```
howard query \
   --explode_infos \
   --explode_infos_prefix='INFO/' \
   --query="SELECT \"#CHROM\", POS, REF, ALT, GROUP_CONCAT(INFO, ';') AS INFO \
            FROM parquet_scan('tests/databases/annotations/current/hg19/*.parquet', union_by_name = true) \
            GROUP BY \"#CHROM\", POS, REF, ALT" \
   --output=/tmp/full_annotation.tsv

head -n2 /tmp/full_annotation.tsv
```

Expected result (depends on Parquet databases)

```
#CHROM  POS     REF     ALT     INFO
chr1    69093   G       T       MCAP13=0.001453;REVEL=0.117;SIFT_score=0.082;SIFT_converted_rankscore=0.333;S
```

How to explore genetics variations from VCF files?

CuteVariant: A standalone and free application to explore genetics variations from VCF files

Cutevariant is a cross-plateform application dedicated to maniupulate and filter variation from annotated VCF file. When you create a project, data are imported into an sqlite database that cutevariant queries according your needs. Presently, SnpEff and VEP annotations are supported. Once your project is created, you can query variant using different gui controller or directly using the VQL language. This Domain Specific Language is specially designed for cutevariant and try to keep the same syntax than SQL for an easy use.

Published in Bioinformatics Advanced - Cutevariant: a standalone GUI-based desktop application to explore genetic variations from an annotated VCF file

Documentation available on cutevariant.labsquare.org and GitHub

CuteVariant